

On the Necessity of Public Contexts in Hybrid KEMs: A Case Study of X-Wing

T. Kang, C. Lee, and Y. Son

Abstract—Post-quantum migration must balance two risks: future quantum breaks of classical cryptography and residual uncertainty in newly standardized post-quantum cryptography (PQC). Hybrid Key Encapsulation Mechanisms (KEMs) hedge by combining a classical and a PQC component. Prior work shows that optimized combiners may omit large public inputs from the final key-derivation step, but only if the derived key remains bound to the ciphertext transcript and, in multi-target settings, to the intended recipient; otherwise ciphertext manipulation and cross-recipient amortization at the KDF layer can increase an adversary’s concrete advantage. In practice, these requirements are often conflated, leading either to unsafe secret-only schedules or to unnecessary hashing of large transcripts. We distill practitioner-facing, interface-level guidance by separating ciphertext-to-secret binding from multi-target security, and by adopting ciphertext second-preimage resistance (C2PRI) as a checkable criterion under deployed encodings.

We apply this perspective to X-Wing, a hybrid combining ML-KEM with an X25519-based DH-to-KEM under consideration as an IETF Internet-Draft. Under the deployed raw-output interface, we show how distinct classical ciphertexts can yield the same shared secret, motivating hashing of the classical ciphertext in the outer KDF and clarifying when recipient public-key context is needed in multi-target deployments. We also show that similar issues arise for other widely deployed elliptic-curve Diffie–Hellman (ECDH) APIs, including P-256, when they export only partial point information. Finally, we summarize when ciphertext hashing can be omitted, including canonical prime-order abstractions such as Ristretto255 and designs that internalize transcript context via per-component hashing, as in HPKE Diffie–Hellman-based KEM (DHKEM) profiles.

Index Terms—C2PRI, Hybrid KEM, Post-quantum cryptography, X-Wing.

I. INTRODUCTION

A KEY Encapsulation Mechanism (KEM) is a primary building block for establishing session keys in modern secure channels, enabling efficient symmetric encryption for bulk data. The long-term confidentiality of sensitive communications—especially archival data—therefore depends critically on the secrecy of the session keys they establish.

This dependence becomes acute in the post-quantum setting. Once large-scale quantum computers capable of running Shor’s algorithm [1] become available, conventional public-key mechanisms based on factoring and discrete logarithms

will no longer provide adequate security. The urgency is amplified by Harvest-Now-Decrypt-Later (HNDL) threats, where adversaries store encrypted traffic today and decrypt it later. In many deployments, a future failure of the key-establishment mechanism retroactively compromises previously captured ciphertexts, making KEM transition a central priority for post-quantum cryptography (PQC) migration.

However, relying solely on newly standardized PQC introduces a different class of risk: it concentrates long-term confidentiality on primitives that have not yet benefited from decades of cryptanalytic scrutiny. Recent cryptanalytic breaks of seemingly mature candidates (e.g., SIKE [2]) demonstrate that community confidence can collapse abruptly, even late in the standardization pipeline.

Beyond theoretical hardness, initial large-scale deployments encounter practical implementation threats, notably side-channel attacks and unpredictable engineering challenges. For high-assurance infrastructure, treating a single standardized PQC primitive as the sole defense remains a precarious strategy.

These considerations motivate *hybrid KEMs*, which combine a well-studied classical component with a PQC component so that confidentiality is preserved even if one component is later weakened. A natural starting point is to hash only the two shared secrets. However, many deployed designs require additional binding: the final key should be bound to the public transcript and, in multi-target settings, to the intended recipient. This raises a design choice about where binding is enforced—within the components or at the external combiner. A substantial body of work studies this trade-off, ranging from conservative hash-everything designs to optimized combiners that omit public context when component-level binding suffices [3]–[6].

We illustrate three representative hybrid key schedules, differing in which public inputs are absorbed by the final key-derivation function (KDF); we omit a fixed domain-separation label for readability. Here $i \in \{1, 2\}$ indexes the PQC ($i = 1$) and classical ($i = 2$) components, with K_i the component shared secret and (c_i, pk_i) the ciphertext and public key when included:

- Secret-only:

$$K = H(K_1 \| K_2).$$

- X-Wing (simplified) [6]:

$$K = H(K_1 \| K_2 \| c_2 \| pk_2).$$

T. Kang and C. Lee are with the School of Cybersecurity, Korea University, Seoul 02841, South Korea (e-mail: {kth9999, changminlee}@korea.ac.kr).

Y. Son is with the Department of Convergence Security Engineering, Sungshin Women’s University, Seoul 02844, South Korea (e-mail: yongha.son@sungshin.ac.kr).

T. Kang and C. Lee contributed equally to this work and should be considered joint first authors.

Corresponding author: Y. Son, e-mail: yongha.son@sungshin.ac.kr.

- Hash-everything (Giacon et al. [3]):

$$K = H(K_1 \| K_2 \| c_1 \| c_2 \| pk_1 \| pk_2).$$

In this work, to provide precise guidance on what can be safely omitted, we separate two orthogonal requirements: ciphertext-to-secret binding, captured by ciphertext second-preimage resistance (C2PRI; Definition 3), and multi-target confidentiality in multi-user deployments, captured by the standard multi-target IND-CCA notion (MT-IND-CCA; Definition 4). Intuitively, C2PRI rules out distinct ciphertext transcripts that decapsulate to the same shared secret for a fixed recipient, while MT-IND-CCA captures the generic N -fold amplification that arises when an adversary may target any one of N recipients.

In hybrid combiners, explicitly including a recipient identifier (e.g., pk_2) in the outer KDF acts as domain separation and can mitigate this amplification at the KDF layer, without changing the underlying IND-CCA security of the component KEMs.

Distinguishing these two properties is crucial for safe implementation. It prevents practitioners from insecurely defaulting to secret-only key derivation when public context is essential for binding, while simultaneously avoiding the efficiency penalty of unnecessarily hashing large transcripts when the primitive already guarantees binding internally.

We use X-Wing [7] as a running case study. X-Wing combines ML-KEM with an X25519-based Diffie–Hellman (DH)-to-KEM component and prunes the outer KDF input by omitting the large PQC ciphertext. On the classical side, X-Wing exports the raw X25519 output, making the necessity of hashing c_2 (for C2PRI) and the role of pk_2 (as a multi-target domain separator) sensitive to the deployed interface. However, the specification includes these inputs; we clarify the concrete failure modes that motivate them.

A. Our Contributions

Our analysis clarifies when these external transcript inputs become redundant by tying them to two orthogonal, interface-level questions: whether the deployed decapsulation interface is injective from ciphertext transcripts to shared secrets (captured by C2PRI), and whether the resulting key schedule achieves explicit recipient domain separation in multi-target settings.

We first show that the Raw X25519 KEM is non-injective under the deployed x -only interface due to Curve25519’s cofactor/quotient structure, so distinct classical ciphertext transcripts can decapsulate to the same shared secret; this motivates hashing the classical ciphertext in the outer combiner. We then highlight that group-level properties alone can be misleading under deployed APIs: for widely deployed P-256 ECDH interfaces that export only an x -coordinate-derived output, distinct shared points can induce the same exported secret, so C2PRI can still fail despite prime order.

Next, we identify classical instantiations whose deployed interfaces are binding-friendly—in particular, canonical prime-order abstractions such as Ristretto255 [8], [9]—which can

justify omitting c_2 from the external combiner. Finally, we discuss an orthogonal approach that internalizes both transcript and (optionally) recipient context within the classical component via per-component hashing of the DH value together with public inputs. HPKE-style DHKEM profiles follow this design [10], enabling an external combiner that can omit c_2 and, when recipient context is internalized, also omit pk_2 ; otherwise, explicit inclusion of pk_2 remains necessary to mitigate multi-target amplification in multi-user deployments. Specifically, we make the following contributions:

- **Non-injectivity of deployed x -only DH-to-KEM interfaces:** We show that the Raw X25519 KEM fails C2PRI due to Curve25519’s cofactor/quotient structure, and that widely deployed x -only ECDH outputs such as P-256 remain non-injective. These results explain why X-Wing must retain the classical ciphertext to recover ciphertext-to-secret binding under mainstream interfaces.
- **When the outer combiner can omit the classical ciphertext c_2 :** We give interface-level criteria (via C2PRI) for deciding whether c_2 must be included in the outer KDF, and we identify binding-friendly instantiations that justify omitting it, most notably canonical prime-order abstractions such as Ristretto255 [8], [9].
- **Multi-target confidentiality and when the outer combiner can omit pk_2 :** We clarify how omitting recipient-specific context affects the concrete multi-target advantage in multi-user deployments, and how including pk_2 serves as domain separation for the outer KDF. We also discuss when this becomes redundant because the key schedule already hashes a per-target public salt, or because recipient separation is already internalized at the component or protocol level.

Table I summarizes these conditions for representative classical components, indicating which public context values must remain in the external combiner.

TABLE I
EXTERNAL CONTEXT TYPICALLY INCLUDED BY THE HYBRID COMBINER FOR THE CLASSICAL COMPONENT (UNDER TYPICAL DEPLOYED INTERFACES).

Classical component	Include c_2 ?	Include pk_2 ?
The Raw X25519 KEM (X-Wing classical side)	Yes	Yes
P-256 ECDH (standard x-only output)	Yes	Yes
Ristretto255 (canonical interface)	No	Yes
HPKE-style DHKEM	No	No

B. Related Work

The concept of hybridizing classical and post-quantum schemes has been explored extensively in both academia and practice.

Theoretical foundations of hybrid combiners. Giacon et al. [3] formalized robust KEM combiners (the hash-everything

combiner of Giacon et al.), advocating a conservative paradigm in which the final KDF absorbs all relevant public transcript material (shared secrets, ciphertexts, and public keys). This hash-everything approach provides strong generic robustness guarantees, but it can be computationally costly when one component has large ciphertexts, motivating subsequent work on optimized combiners that omit selected public inputs under additional component-level assumptions.

Optimized combiners and applicability analyses. X-Wing [6] is a prominent recent proposal that combines X25519 and ML-KEM and exploits ML-KEM’s properties to omit the large post-quantum ciphertext from the final KDF input. Follow-up analyses study when such pruning is justified and how broadly X-Wing-style design choices apply across components and deployed interfaces [5], [11]. Our work complements this direction by pinpointing interface-level binding conditions for classical components and by explaining where X25519-based raw DH-to-KEM interfaces (and more generally raw DH-derived KEMs) fail due to cofactor/quotient effects and multi-target security requirements.

Early adopters and experimental hybrids. Early deployments explored hybrid key exchange largely as pragmatic risk hedging rather than as instances of a single, standardized combiner template. OpenSSH [12] introduced hybrid SSH key exchanges (e.g., X25519 with NTRU Prime and later ML-KEM) to future-proof sessions while retaining well-understood classical mechanisms. Similarly, Google experimented in Chrome with CEC PQ1 (X25519 + NewHope) [13] and CEC PQ2 (X25519 + HRSS/NTRU) [14]. These efforts predate (and need not match) the exact X-Wing-style key schedule; instead, transcript handling is dictated by each protocol’s context and threat model. More recent deployments increasingly move toward standardized hybrid profiles in TLS-related settings [10].

C. Organization

The remainder of this paper is organized as follows. Section II introduces our deployed-interface model via representation functions and recalls the security notions needed to reason about public-context omission, in particular IND-CCA, C2PRI, and multi-target IND-CCA. Section III analyzes classical components where C2PRI fails under mainstream deployed interfaces—including the Raw X25519 KEM and P-256 with x -only outputs—showing that the classical ciphertext c_2 must be included in the outer KDF to prevent ciphertext substitution. Section IV identifies classical instantiations for which c_2 becomes redundant, either because the deployed interface is injective over a prime-order abstraction like Ristretto255 or because transcript context is internalized by a per-component KDF such as HPKE-style DHKEM). Section V studies multi-target confidentiality and clarifies the role of pk_2 as a domain separator that mitigates multi-user amplification effects in hybrid combiners, and Section VI concludes.

II. PRELIMINARIES

A. Notation

We use standard cryptographic notation. Algorithms are written in sans-serif (e.g., KeyGen), and sets/spaces in calligraphic font (e.g., $\mathcal{K}$). Because our results depend on the concrete byte representations used in deployments, we distinguish mathematical group elements from the byte strings that implementations transmit as ciphertexts and export as shared secrets. We write λ for the security parameter, and $\text{negl}(\lambda)$ for an arbitrary negligible function of λ , i.e., for every polynomial $p(\cdot)$ there exists λ_0 such that for all $\lambda \geq \lambda_0$, $\text{negl}(\lambda) \leq 1/p(\lambda)$.

TABLE II
SUMMARY OF NOTATIONS

Symbol	Description
$x \xleftarrow{\$} S$	Uniform random sampling of x from set S
$\parallel$	Concatenation of bitstrings
$\text{negl}(\lambda)$	A negligible function in the security parameter λ
$\mathbb{F}_p$	Finite field of prime order p
$E(\mathbb{F}_p)$	Group of $\mathbb{F}_p$ -rational points on an elliptic curve E
$\mathcal{O}$	Identity element of an elliptic-curve group
$E[n]$	n -torsion subgroup $\{T \in E(\mathbb{F}_p) : [n]T = \mathcal{O}\}$
h, q	Cofactor and large prime with $ E(\mathbb{F}_p) = hq$
$[k]P$	Scalar multiplication of point P by scalar k
sk, pk	Secret key and public key
c, K	Ciphertext (public transcript) and shared secret
$\text{Rep}_c, \text{Rep}_K$	Deployed transcript / shared-secret representations (Definition 1)
$\mathcal{SS}$	Shared secret space of the KEM
$\text{X25519}(k, x)$	RFC 7748 X25519: Montgomery ladder with clamped scalar k on input x

B. Deployed Interfaces and Injectivity

Our binding criteria depend on the concrete strings exported by a deployed interface: lossy representations (e.g., x -only outputs that identify P and $-P$) can break the conditions needed to omit c_2 even when the underlying group is well behaved. We therefore model (i) how ephemeral public values are represented as ciphertexts and (ii) how DH shared group elements are represented as decapsulation outputs via explicit representation functions.

Definition 1 (Injective deployed interface). A DH-style component induces two deterministic representation functions:

$$\text{Rep}_c : \mathcal{C} \rightarrow \{0, 1\}^*, \quad \text{Rep}_K : \mathcal{Z} \rightarrow \mathcal{SS},$$

where $\mathcal{C}$ is the set of public group elements accepted as ciphertexts and $\mathcal{Z}$ is the set of DH shared group elements that can arise from such ciphertexts. Here Rep_c serializes ciphertexts, and Rep_K specifies the exported format of the shared secret.

We call the deployed interface *injective (for binding)* if Rep_c is injective on $\mathcal{C}$ and Rep_K is injective on $\mathcal{Z}$.

C. Curve25519 and X25519

We recall only the facts used in this paper. Curve25519 [15] is the Montgomery curve $y^2 = x^3 + 486662x^2 + x$ over $\mathbb{F}_p$ with $p = 2^{255} - 19$. Its group order is $|E(\mathbb{F}_p)| = 8q$ where q is a large prime; thus the cofactor is $h = 8$.

X25519 (RFC 7748 [16]) implements scalar multiplication using a Montgomery ladder that depends only on the input x -coordinate and performs a fixed sequence of field operations, enabling constant-time implementations. Two properties matter for our later binding analysis: (i) clamping forces secret scalars to be multiples of 8, and (ii) the standard X25519 decapsulation interface is validation-free in the sense that it accepts any 32-byte string as an input x -coordinate and returns an x -coordinate output.

1) *Scalar clamping*: RFC 7748 clamps a raw 32-byte scalar k as follows:

$$k \leftarrow (k \wedge 248); \quad k[31] \leftarrow (k[31] \wedge 127) \vee 64.$$

The first operation clears the lowest three bits, enforcing

$$k \equiv 0 \pmod{8},$$

which (among other engineering benefits) ensures that scalar multiplication annihilates any small-subgroup component of order dividing 8. The second operation fixes the effective bit-length to ensure a fixed ladder length.

2) *Validation-free x -coordinate interface*: A deployed X25519 public key is a field element $pk \in \mathbb{F}_p$ represented as a 32-byte string, interpreted as an x -coordinate. Given a clamped secret scalar sk and a peer value $x \in \mathbb{F}_p$, X25519 outputs another field element with Montgomery ladder

$$K = \text{X25519}(sk, x) \in \mathbb{F}_p,$$

which is used as an x -coordinate output. When x corresponds to a valid Curve25519 point P , this function outputs the x -coordinate of the scalar multiple $[sk]P$; the deployed interface, however, accepts any 32-byte string as input and returns the resulting field element.

D. Security Definitions

We recall IND-CCA security and define the additional notions needed to reason about public-context omission in hybrid combiners, in particular C2PRI and multi-target IND-CCA security.

1) *IND-CCA security*: We recall the standard IND-CCA notion as our baseline confidentiality definition for KEMs against an active adversary with decapsulation-oracle access.

Definition 2 (IND-CCA Security). A KEM is IND-CCA secure if for any PPT adversary $\mathcal{A}$,

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr[\text{IND-CCA}_{\text{KEM}}(\mathcal{A}) = 1] - \frac{1}{2} \right|$$

is negligible where the game is given in Algorithm 1.

Algorithm 1 IND-CCA game for a KEM

```

1:  $(sk, pk) \leftarrow \text{KeyGen}(1^\lambda)$ 
2:  $(c^*, K_0^*) \leftarrow \text{Encaps}(pk)$ 
3:  $K_1^* \xleftarrow{\$} \mathcal{SS}$ 
4:  $b \xleftarrow{\$} \{0, 1\}$ 
5:  $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{dec}}}(pk, c^*, K_b^*)$ 
6: Oracle  $\mathcal{O}_{\text{dec}}(c)$ :
7:   if  $c = c^*$  then return  $\perp$ 
8:   else return  $\text{Decaps}(sk, c)$ 
9: return 1 if  $b' = b$ , else 0

```

2) *Ciphertext-to-secret binding (C2PRI)*: C2PRI captures *ciphertext-to-secret binding* for a fixed recipient: it rules out distinct ciphertexts that decapsulate to the same shared secret under the same secret key. This property is crucial for optimized hybrid KEMs where a component ciphertext c_i is omitted from the external combiner. Without C2PRI, an adversary could substitute c_i with a distinct c'_i that yields the same secret. Formally, such malleability allows an adversary to distinguish the real shared secret from random by submitting the modified ciphertext c'_i to a decapsulation oracle, thereby violating IND-CCA security.

Definition 3 (C2PRI). A KEM satisfies C2PRI if for any PPT adversary $\mathcal{A}$,

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) = \Pr[\text{C2PRI}_{\text{KEM}}(\mathcal{A}) = 1]$$

is negligible, where the game is given in Algorithm 2.

Algorithm 2 C2PRI game for a KEM

```

1:  $(sk, pk) \leftarrow \text{KeyGen}(1^\lambda)$ 
2:  $(c^*, K^*) \leftarrow \text{Encaps}(pk)$ 
3:  $c' \leftarrow \mathcal{A}(pk, c^*, K^*)$ 
4: return 1 if  $c' \neq c^*$  and  $\text{Decaps}(sk, c') = K^*$ , else 0

```

3) *Multi-target confidentiality (multi-user setting)*: In a multi-user deployment, a KEM is instantiated for many recipients, each with an independently generated key pair. A *multi-target* adversary succeeds if it compromises the confidentiality of *any* one recipient among these targets. Compared to the single-recipient setting, this leads to a generic amplification effect: if an adversary has advantage $\varepsilon(\lambda)$ against one target, then against N independent targets its success probability can grow to approximately $N \cdot \varepsilon(\lambda)$.

This notion is formalized by multi-target (a.k.a. multi-user) IND-CCA security.

Definition 4 (Multi-target IND-CCA security). A KEM is multi-target IND-CCA secure if for any PPT adversary $\mathcal{A}$ and any polynomially bounded number of targets $N = N(\lambda)$,

$$\text{Adv}_{\text{KEM}, N}^{\text{MT-IND-CCA}}(\mathcal{A}) = \left| \Pr[\text{MT-IND-CCA}_{\text{KEM}, N}(\mathcal{A}) = 1] - \frac{1}{2} \right|$$

is negligible, where the game is given in Algorithm 3.

Algorithm 3 MT-IND-CCA game for a KEM with N targets

```
1:  $b \xleftarrow{\$} \{0, 1\}$ 
2: for  $j = 1$  to  $N$  do
3:    $(sk_j, pk_j) \leftarrow \text{KeyGen}(1^\lambda)$ 
4:    $(c_j^*, K_{j,0}^*) \leftarrow \text{Encaps}(pk_j)$ 
5:    $K_{j,1}^* \xleftarrow{\$} \mathcal{SS}$ 
6: end for
7:  $\tilde{pk} \leftarrow (pk_1, \dots, pk_N)$ 
8:  $\tilde{c}^* \leftarrow (c_1^*, \dots, c_N^*)$ 
9:  $\tilde{K}_b^* \leftarrow (K_{1,b}^*, \dots, K_{N,b}^*)$ 
10:  $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{dec}}}(pk, \tilde{c}^*, \tilde{K}_b^*)$ 
11: Oracle  $\mathcal{O}_{\text{dec}}(j, c)$ :
12:   if  $c = c_j^*$  then return  $\perp$ 
13:   else return  $\text{Decaps}(sk_j, c)$ 
14: return 1 if  $b' = b$ , else 0
```

E. Cofactors and Kernels

Many elliptic-curve groups used in practice have a non-trivial cofactor $h > 1$, i.e., $|E(\mathbb{F}_p)| = hq$ for a large prime q . Let $E[q]$ denote the prime-order subgroup and $E[h]$ the h -torsion subgroup. We will use the following kernel phenomenon to explain why some raw DH-derived KEM interfaces admit ciphertext second-preimages (violating C2PRI).

A key point is that scalar multiplication by a secret scalar d may have a non-trivial kernel on torsion:

$$\ker([d]) \cap E[h] \neq \{\mathcal{O}\} \quad \text{whenever} \quad \gcd(d, h) \neq 1.$$

In that case, for any $P \in E(\mathbb{F}_p)$ and non-trivial $T \in \ker([d]) \cap E[h]$,

$$[d](P + T) = [d]P,$$

so the map $[d]$ is not injective at the curve-point level. While this can be benign for basic DH secrecy, it is harmful for binding properties such as C2PRI for raw DH-derived KEM instantiations, since distinct public inputs can induce the same shared group element (and hence the same exported secret under non-injective representations). In Section IV-A we return to a standard interface-level mitigation that avoids cofactor-induced ambiguity by exposing a canonical prime-order abstraction.

III. WHEN c_2 MUST BE INCLUDED: FAILURES OF C2PRI

In optimized hybrid KEMs, one may wish to omit the classical ciphertext c_2 from the external combiner. This omission is safe for ciphertext-to-secret binding only if the classical component is C2PRI under the deployed ciphertext-to-secret interface (cf. Definition 3).

However, many classical KEMs are instantiated as *Raw DH-derived KEMs*, where the shared secret is the raw Diffie-Hellman value without mixing in the ciphertext. In such constructions, implementation details often lead to C2PRI failures. We analyze two prominent examples where this occurs: X25519 and NIST P-256.

A. The Raw X25519 KEM under RFC 7748

X25519 is widely deployed and is considered secure as a DH function. However, when used as a component inside a hybrid KEM, one must additionally account for binding: the derived component secret should be uniquely tied to the public transcript in the sense required by robust hybrid analyses. X-Wing instantiates its classical side using the raw approach: it publishes an ephemeral X25519 x -coordinate as the ciphertext and exports the bare DH output as the shared secret.

Algorithm 4 formalizes this deployed ciphertext-to-secret interface. In encapsulation, the sender samples an ephemeral clamped scalar sk_E , outputs the ephemeral public value $c \leftarrow \text{X25519}(sk_E, 9)$ as the ciphertext, and exports the shared secret as the raw DH output $K_2 \leftarrow \text{X25519}(sk_E, pk)$. Decapsulation on input c returns the same raw DH value $K_2 \leftarrow \text{X25519}(sk, c)$, without any per-component KDF that hashes in (c, pk) . The constant 9 denotes the standard Curve25519 base-point x -coordinate.

Algorithm 4 Raw X25519 KEM

```
1: Function KeyGen( $1^\lambda$ ):
2:    $k \xleftarrow{\$} \{0, 1\}^{256}$  // 32-byte string
3:    $k[0] \leftarrow k[0] \wedge 248, k[31] \leftarrow (k[31] \wedge 127) \vee 64$ 
4:    $sk \leftarrow k$ 
5:    $pk \leftarrow \text{X25519}(sk, 9)$ 
6:   return  $(sk, pk)$ 
7:
8: Function Encaps( $pk$ ):
9:    $k_E \xleftarrow{\$} \{0, 1\}^{256}$  // 32-byte string
10:   $k_E[0] \leftarrow k_E[0] \wedge 248, k_E[31] \leftarrow (k_E[31] \wedge 127) \vee 64$ 
11:   $sk_E \leftarrow k_E$ 
12:   $c \leftarrow \text{X25519}(sk_E, 9)$ 
13:   $K_2 \leftarrow \text{X25519}(sk_E, pk)$ 
14:  return  $(c, K_2)$ 
15:
16: Function Decaps( $sk, c$ ):
17:   $K_2 \leftarrow \text{X25519}(sk, c)$ 
18:  return  $K_2$ 
```

B. Formal Failure of C2PRI for the Raw X25519 KEM

The ciphertext second-preimage attack exploits the algebraic structure of Montgomery curves. Specifically, for any point $P = (x, y)$ on the curve $y^2 = x^3 + Ax^2 + x$, adding the unique 2-torsion point $T = (0, 0)$ results in a point with the inverted coordinate $x^{-1} \pmod{p}$. Thus, an adversary can efficiently construct a collision simply by computing the modular inverse of the ciphertext: $c' = c^{-1} \pmod{p}$.

Remark 1 (Why the shared secret remains invariant). The reason this distinct ciphertext c' decapsulates to the same key lies in the definition of the X25519 scalar. X25519 scalars are "clamped" to be multiples of the cofactor $h = 8$. Since T is a point of order 2, it is annihilated by the scalar multiplication: $[sk]T = \mathcal{O}$. Consequently, decoding the modified ciphertext

yields $[sk](P + T) = [sk]P + [sk]T = [sk]P$, producing the identical shared secret.

We formalize the binding failure for the Raw X25519 KEM. RFC 7748 clamping forces the effective scalar to be a multiple of 8, which induces a cofactor-related collision structure that breaks ciphertext-to-secret binding.

Theorem 1 (C2PRI Failure of the Raw X25519 KEM). *There exists a PPT adversary that wins the C2PRI game against the Raw X25519 KEM with probability $1 - \text{negl}(\lambda)$.*

Proof: We construct an explicit PPT adversary $\mathcal{A}$ that wins the C2PRI game.

Step 1 (Define the second-preimage candidate). On input (pk, c^*, K^*) , $\mathcal{A}$ interprets c^* as a field element in $\mathbb{F}_p$ (RFC 7748 decoding), computes the modular inverse, and outputs

$$c' := (c^*)^{-1} \pmod{p}.$$

Except with negligible probability over Encaps , we have $c^* \neq 0$ so the inversion is defined, and $c' \neq c^*$.

Step 2 (Relate inversion to a torsion shift). Let $E(\mathbb{F}_p)$ denote Curve25519 and let $T = (0, 0) \in E(\mathbb{F}_p)$ be the non-trivial 2-torsion point. Choose any point $P^* \in E(\mathbb{F}_p)$ with $x(P^*) = c^*$ and set $P' := P^* + T$. $x(P') = x(P^* + T) = x(P^*)^{-1} = c'$, so c' corresponds to the torsion-shifted public value at the deployed x -only interface.

Step 3 (Show the decapsulation collision). RFC 7748 clamping enforces $sk \equiv 0 \pmod{8}$, hence $[sk]T = \mathcal{O}$ because T has order 2. Therefore,

$$\begin{aligned} \text{Decaps}(sk, c') &= x([sk]P') \\ &= x([sk](P^* + T)) \\ &= x([sk]P^* + [sk]T) \\ &= x([sk]P^*) \\ &= \text{Decaps}(sk, c^*) \\ &= K^*. \end{aligned}$$

Thus $c' \neq c^*$ yet $\text{Decaps}(sk, c') = K^*$, so $\mathcal{A}$ wins the C2PRI game with probability $1 - \text{negl}(\lambda)$. ■

Consequently, because the Raw X25519 KEM fails C2PRI, a secret-only external combiner remains blind to ciphertext substitution. To address this, and in line with analyses of optimized combiners, such as Connolly et al. [5], the external KDF must explicitly include c_2 to enforce ciphertext-to-secret binding. X-Wing adopts this countermeasure by hashing the classical ciphertext c_2 alongside the shared secrets.

At this point, one might conclude that the problem is purely cofactor-related—and therefore that moving to a prime-order curve would automatically restore C2PRI. However, this intuition is misleading at the deployed-interface level: even for prime-order groups, C2PRI can fail if the exported shared-secret representation is non-injective.

C. Prime order is not enough: NIST curve with x -only output

In particular, for NIST curves such as P-256, ECDH interfaces typically export only the x -coordinate of the shared

point, and the points $Q = (x, y)$ and $-Q = (x, -y)$ share the same x -coordinate.

Concretely, consider a raw P-256 KEM that publishes a standard point encoding as ciphertext and outputs

$$K_2 \leftarrow x([sk]P)$$

as the shared secret. At the interface level, Rep_c is injective over valid points, but $\text{Rep}_K(Q) = x(Q)$ is not injective. Given a challenge ciphertext $c^* = \text{Rep}_c(P^*)$, an adversary can output

$$c' := \text{Rep}_c(-P^*),$$

obtaining $c' \neq c^*$ but

$$\begin{aligned} \text{Decaps}(sk, c') &= x([sk](-P^*)) \\ &= x(-[sk]P^*) \\ &= x([sk]P^*) \\ &= \text{Decaps}(sk, c^*), \end{aligned}$$

so C2PRI fails under the standard x -only ECDH-style output.

Implication for hybrid combiners. Thus, even with prime order, mainstream x -only deployed outputs do not by themselves justify omitting c_2 . In such settings, ciphertext-to-secret binding must be enforced elsewhere—either by keeping c_2 in the external combiner, or by switching to a construction that internalizes binding like HPKE-style DHKEM.

IV. WHEN c_2 CAN BE OMITTED: ROBUST INTERFACES

We now discuss classical-component choices for which the obstacle above is absent under the deployed interface (Definition 1), so the hybrid combiner may omit the classical ciphertext c_2 from its final KDF without relying on the combiner to repair representation-level non-injectivity. In practice, c_2 is redundant for ciphertext-to-secret binding via two standard routes: (i) binding-friendly deployed interface, which is injective and prime-order as exposed, and (ii) internalized binding like HPKE-style DHKEM.

Lemma 1 (Prime-order raw DH-KEM is C2PRI under an injective interface). *Let $\mathbb{G}$ be a cyclic group of prime order q . Consider a raw DH-based KEM where the ciphertext and decapsulation output are representations of group elements $P \in \mathbb{G}$ and $[sk]P \in \mathbb{G}$ via functions Rep_c and Rep_K , respectively.*

If both representation functions Rep_c and Rep_K are injective over their valid domains, and KeyGen samples $sk \in \mathbb{Z}_q^$, then the KEM satisfies C2PRI (Definition 3).*

Proof: Consider any adversary $\mathcal{A}$ in the C2PRI game that outputs a valid ciphertext c' given (pk, c^*, K^*) . Suppose the decapsulation succeeds with the same key, i.e., $\text{Decaps}(sk, c') = \text{Decaps}(sk, c^*)$. Let $P', P^* \in \mathbb{G}$ be the group elements corresponding to c' and c^* .

The collision in decapsulation output implies $\text{Rep}_K([sk]P') = \text{Rep}_K([sk]P^*)$. By the injectivity of Rep_K , this requires $[sk]P' = [sk]P^*$ in $\mathbb{G}$. Since $\mathbb{G}$ is of prime order q and $sk \in \mathbb{Z}_q^*$, the scalar multiplication by sk is a bijection, implying $P' = P^*$.

Finally, the injectivity of Rep_c implies that $c' = \text{Rep}_c(P') = \text{Rep}_c(P^*) = c^*$. Since any valid ciphertext yielding the same key must be identical to c^* , no distinct second preimage exists. Therefore, $\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) = 0$. ■

A. Prime-order abstractions as exposed: Ristretto255

Ristretto255 removes cofactor-induced ambiguity at the *interface* level by working with a prime-order abstraction: informally, it identifies curve points that differ by the cofactor torsion and exposes a canonical representative for each resulting abstract element [8], [9]. As a consequence, both ciphertexts and shared secrets can be represented via unique canonical encodings, yielding an injective deployed interface for binding.

Let $\mathbb{G}_{\text{rist}}$ denote the prime-order group induced by Ristretto255 and let $\mathcal{R}$ denote the set of valid canonical encodings. By design, decoding is well-defined on $\mathcal{R}$ and each group element has a unique encoding in $\mathcal{R}$. Thus the ciphertext representation Rep_c is injective over accepted ciphertexts. Moreover, if the shared secret is exported as the canonical encoding of the shared element in $\mathbb{G}_{\text{rist}}$, then Rep_K is also injective over shared values induced by accepted ciphertexts. Since $\mathbb{G}_{\text{rist}}$ has prime order, the conditions of Lemma 1 hold as deployed, and a raw DH-derived KEM instantiation over Ristretto255 is C2PRI.

Implication for hybrid combiners. When the classical component is instantiated over Ristretto255 using the standard canonical interface for both ciphertexts and shared secrets, the classical ciphertext c_2 is redundant for ciphertext-to-secret binding and may be omitted from the external hybrid combiner.

B. Internalizing binding: HPKE-style DHKEM constructions

In contrast to the raw DH constructions discussed above, HPKE-style mechanisms do not export the bare Diffie-Hellman value as the shared secret. Instead, they derive the component key by applying a KDF to the raw DH result mixed with the encapsulation transcript and other public context. Abstractly:

$$Z \leftarrow \text{DH}(sk, c), \quad K_2 \leftarrow \text{KDF}(Z \parallel c \parallel \text{ctx}),$$

where ctx denotes auxiliary context (typically including the pk).

Crucially for C2PRI, any transcript substitution $c \mapsto c'$ changes the KDF input. Therefore, producing the same K_2 for distinct ciphertexts ($c' \neq c$) would require breaking the (modeled) pseudorandomness of the internal KDF.

Implication for hybrid combiners. Since HPKE-style DHKEM internalizes c_2 through its internal KDF, the classical component output K_2 is already transcript-bound. Accordingly, an external hybrid combiner can omit c_2 on the classical side without sacrificing ciphertext-to-secret binding. Furthermore, as ctx often binds the public key, such constructions may also allow omitting pk_2 externally, as discussed in Section V.

V. MULTI-TARGET CONFIDENTIALITY AND THE ROLE OF pk_2

The criteria above address ciphertext-to-secret binding and determine when the classical ciphertext c_2 can be omitted from the external combiner without enabling same-recipient ciphertext substitution. We now turn to an orthogonal concern: *multi-target confidentiality* in multi-user deployments.

In real-world settings, a hybrid KEM is instantiated for many recipients. A multi-target adversary succeeds if it compromises the confidentiality of *any* one recipient among these targets. Even when the underlying single-target KEM is IND-CCA secure, the adversary's overall success probability can grow linearly with the number of recipients N . Here we focus on an additional, avoidable source of amplification that arises when the outer KDF domain is shared across recipients, enabling cross-target amortization beyond the generic multi-user union bound.

In hybrid constructions with an external combiner, this amplification becomes particularly relevant when recipient-specific public context is omitted from the final key-derivation function (KDF). If the classical component secret K_2 does not internalize recipient identity, then all recipients share the same key-derivation domain, allowing adversarial work to be reused across targets.

A. The public key as a salt for multi-target security

Including the recipient public key pk_2 (or a fixed-length identifier derived from it) in the external combiner acts as a *domain separator* (or salt) for the KDF. This ensures that each recipient's derived key is sampled from an independent domain, preventing cross-recipient reuse of adversarial effort and restoring the standard single-target security bound per recipient.

This role of pk_2 is conceptually different from ciphertext-to-secret binding. Even when C2PRI holds and ciphertext substitution is impossible, omitting pk_2 can still weaken confidentiality in multi-target deployments by enabling generic multi-target amplification effects.

B. Injective classical interfaces do not remove the need for domain separation

Even when the classical component exposes a prime-order injective interface—as Ristretto255 does for C2PRI (Section IV-A)—the role of pk_2 as an external domain separator remains essential. This is because interface injectivity (uniqueness of the shared secret for a fixed recipient) is orthogonal to multi-target domain separation (uniqueness of the KDF domain across recipients).

Lemma 2 (Injectivity alone does not prevent multi-target amplification). *Let H be modeled as a random oracle and let*

$$K_i = H(K_{\text{shared},i})$$

be the derived key for recipient i , where the value $K_{\text{shared},i}$ is injectively derived from the protocol transcript and the recipient's secret key.

Even if the map from transcripts to $K_{\text{shared},i}$ is injective, a multi-target adversary against N recipients gains an $\Theta(N)$ advantage over the single-target setting.

Proof sketch: We write κ for the (conditional) min-entropy of $K_{\text{shared},i}$ given the adversary’s view. Assuming $K_{\text{shared},i}$ has conditional min-entropy at least κ given the adversary’s view, each oracle query targeting a fixed recipient hits with probability at most $2^{-\kappa}$. In the random-oracle model, the adversary succeeds if it queries H on any one of the hidden inputs $K_{\text{shared},1}, \dots, K_{\text{shared},N}$. Each oracle query hits a fixed target with probability at most $1/2^\kappa$. Over Q oracle queries, a union bound yields an overall success probability of at most $NQ/2^\kappa$. Injectivity ensures that the values $K_{\text{shared},i}$ are distinct, but does not prevent this N -fold amplification across recipients. ■

Corollary 1 (Public-key salting removes generic amplification). *If the derived key is computed as*

$$K_i = H(\text{ID}(pk_i) \parallel K_{\text{shared},i}),$$

where $\text{ID}(pk_i)$ are distinct public identifiers, then the adversary’s success probability is bounded by $Q/2^\kappa$, independent of N .

C. Relation to HPKE-style DHKEM

The discussion above explains why injectivity alone is insufficient to control multi-target amplification. HPKE-style DHKEM constructions address this issue by *internalizing recipient context* within the component itself.

Concretely, HPKE-style DHKEM derives the component key by hashing the Diffie–Hellman value together with public context that includes the recipient public key. As a result, the component output is already salted by a per-recipient identifier, and the key-derivation domains are separated across recipients by construction.

Implication for hybrid combiners. When a classical component follows an HPKE-style DHKEM design, explicitly hashing pk_2 again at the external hybrid combiner becomes redundant for multi-target confidentiality. In contrast, for raw DH-derived components such as the Raw X25519 KEM used in X-Wing, the shared secret K_2 does not bind recipient identity, and explicit inclusion of pk_2 at the external combiner remains necessary to mitigate multi-target amplification.

D. When can pk_2 be omitted?

Including pk_2 in the outer KDF is a simple and effective way to domain-separate recipients and tighten concrete multi-target bounds by preventing generic N -fold amplification at the KDF layer. However, pk_2 is not the only possible public salt.

If the outer combiner already hashes a per-target public transcript value that is unique with overwhelming probability, then the KDF input is already domain-separated across recipients in the MT-IND-CCA game, and hashing pk_2 mainly provides conservative redundancy.

Finally, when recipient separation is already internalized at the component or protocol layer—as in HPKE-style DHKEM profiles—explicitly hashing pk_2 again at the external combiner can be safely omitted without degrading multi-target confidentiality.

VI. CONCLUSION

Our main message is that hashing public transcript material in hybrid KEMs is not an intrinsic requirement of *hybridization*; rather, it is a concrete mechanism for enforcing specific binding and domain-separation goals. Accordingly, decisions about omitting public inputs from an external combiner should be made at the *deployed-interface* level, i.e., with respect to the actual byte representations of ciphertexts and exported shared secrets (Definition 1).

First, ciphertext-to-secret binding for a fixed recipient is captured by C2PRI (Definition 3), which governs when the classical ciphertext c_2 may be omitted from an external KDF input. We showed that the Raw X25519 KEM fails C2PRI under the RFC 7748 x -coordinate interface (Theorem 1), enabling efficient ciphertext second-preimages; and we highlighted that prime-order groups do not automatically fix this problem at the API level, since mainstream x -only ECDH outputs are still non-injective due to the P versus $-P$ ambiguity (Section III-C). These results explain why practical hybrids must retain c_2 in the outer KDF whenever the deployed classical interface does not provide transcript binding on its own.

Conversely, we identified when c_2 can be safely omitted without relying on the external combiner to repair primitive-level gaps. Ciphertext omission is justified under binding-friendly deployed interfaces that are injective in both transcript and shared-secret representations over a prime-order abstraction, or when binding is internalized by the component itself via per-component hashing.

Second, concrete multi-target confidentiality in multi-user deployments is captured by the standard multi-target IND-CCA notion (Definition 4). In our setting, omitting recipient-specific context such as pk_2 does not introduce a new *type* of attack but it can allow generic amortization of adversarial effort across N recipients when the outer KDF domain is shared. Including pk_2 (or another recipient identifier) in the outer KDF provides a simple, explicit per-recipient domain separator, while designs that already internalize recipient separation at the component/protocol layer can safely omit pk_2 externally without degrading multi-target confidentiality (Section V).

While pruning public inputs can yield measurable savings when a component ciphertext is large, the primary practical value of our results is *qualitative rather than micro-optimizing*: we provide a deployable checklist that explains *why* a given key schedule includes (or omits) specific public contexts. This reduces the risk of two common failure modes in practice: (i) unsafe secret-only schedules that silently drop necessary binding, and (ii) overly conservative designs that accumulate redundant inputs and protocol state without security benefit.

REFERENCES

- [1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
- [2] W. Castryck and T. Decru, “An Efficient Key Recovery Attack on SIDH,” in *Advances in Cryptology – EUROCRYPT 2023*. Springer, 2023.
- [3] F. Giacon, F. Heuer, and B. Poettering, “Kem combiners,” in *IACR International Workshop on Public Key Cryptography*. Springer, 2018, pp. 190–218.
- [4] J. Alwen, B. Blanchet, E. S. V. Freire, G. George, E. Kiltz, B. Lipp, and D. R. C. M. Mella, “Security of Hybrid Key Encapsulation in the Multiplier Oracle Model,” in *Advances in Cryptology – CRYPTO 2021*. Springer, 2021.
- [5] D. Connolly, K. Hövelmanns, A. Hülsing, S. Kousidis, and M. Meijers, “Starfighters—on the general applicability of x-wing,” *Cryptology ePrint Archive*, 2025.
- [6] D. Connolly, C. Wood *et al.*, “X-Wing: General-purpose Hybrid Post-Quantum KEM,” Internet-Draft, 2024, work in Progress.
- [7] D. Connolly, P. Schwabe, and B. Westerbaan, “X-Wing: general-purpose hybrid post-quantum KEM,” Internet Engineering Task Force, Internet-Draft draft-connolly-cfrg-xwing-kem-09, Sep. 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/09/>
- [8] H. de Valence, I. Lovecruft, and T. Arcieri, “The Ristretto Group,” <https://ristretto.group>, 2019, accessed: 2025-01-01.
- [9] H. Sullivan, N. Sullivan, T. Perrin, and D. Stebila, “Ristretto255 and decaf448,” RFC 9496, nov 2023, iETF. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9496>
- [10] R. Barnes, K. Bhargavan, B. Lipp, and C. Wood, “Hybrid Public Key Encryption,” RFC 9180, 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9180>
- [11] B. Campbell, L. Dao, and A. Moreno, “On key-robustness in hybrid KEMs,” Manuscript/Preprint, 2023.
- [12] OpenSSH Project, “OpenSSH 9.0 Release Notes,” <https://www.openssh.com/txt/release-9.0>, Apr. 2022.
- [13] M. Braithwaite, “Experimenting with post-quantum cryptography,” Google Online Security Blog, <https://security.googleblog.com/2016/07/experimenting-with-post-quantum.html>, Jul. 2016.
- [14] A. Langle, “CECPQ2,” <https://www.imperialviolet.org/2018/12/12/cecpq2.html>, Dec. 2018.
- [15] D. J. Bernstein, “Curve25519: New Diffie-Hellman Speed Records,” in *Public Key Cryptography - PKC 2006*. Springer, 2006, pp. 207–228.
- [16] A. Langle, M. Hamburg, and S. Turner, “Elliptic Curves for Security,” RFC 7748, 2016. [Online]. Available: <https://www.rfc-editor.org/info/rfc7748>